

A Survey on Self-Evolution of Large Language Models

Zhengwei Tao^{1,2}, Ting-En Lin², Xiancai Chen¹, Hangyu Li², Yuchuan Wu², Yongbin Li^{2*},
Zhi Jin^{1*}, Fei Huang², Dacheng Tao³, Jingren Zhou²

¹Key Lab of HCST (PKU), MOE, SCS, Peking University.

²Alibaba Group.

³Nanyang Technological University.

*Corresponding author(s). E-mail(s): shuide.lyb@alibaba-inc.com; zhijin@pku.edu.cn;

Abstract

Large language models (LLMs) have significantly advanced in various fields and intelligent agent applications. However, current LLMs that learn from human or external model supervision are costly and may face performance ceilings as task complexity and diversity increase. To address this issue, self-evolution approaches that enable LLM to autonomously acquire, refine, and learn from experiences generated by the model itself are rapidly growing. This new training paradigm inspired by the human experiential learning process offers the potential to scale LLMs towards superintelligence. In this work, we present a comprehensive survey of self-evolution approaches in LLMs. We first propose a conceptual framework for self-evolution and outline the evolving process as iterative cycles composed of four phases: experience acquisition, experience refinement, updating, and evaluation. Second, we categorize the evolution objectives of LLMs and LLM-based agents; then, we summarize the literature and provide taxonomy and insights for each module. Lastly, we pinpoint existing challenges and propose future directions to improve self-evolution frameworks, equipping researchers with critical insights to fast-track the development of self-evolving LLMs. Our corresponding GitHub repository is available at <https://github.com/AlibabaResearch/DAMO-ConvAI/tree/main/Awesome-Self-Evolution-of-LLM>.[†]

Keywords: Large Language Model, Self-Evolution, Self-Improvement, Self-Training, Autonomous Agents

1 Introduction

With the rapid development of artificial intelligence, large language models (LLMs) like GPT-3.5 [83], GPT-4 [1], Gemini [113], LLaMA [115, 116], and Qwen [8] mark a significant shift in language understanding and generation. These models undergo three stages of development as shown in Figure 1: pre-training on large and diverse corpora to gain a general understanding of language and world knowledge [14, 30], followed by supervised fine-tuning to elicit the abilities of downstream tasks [25, 91]. Finally, the human preference alignment training enables the LLMs to respond as human behaviors [83]. Such successive training paradigms achieve significant breakthroughs, enabling LLMs to perform a wide range of tasks with remarkable zero-shot and in-context capabilities, such as question answering [109], mathematical reasoning [26], code generation [71], and task-solving that require interaction with environments [73].

Despite these advancements, humans anticipate that the emerging generation of LLMs can be tasked with assignments of greater complexity, such as scientific discovery [81] and future events forecasting [95]. However, current LLMs encounter challenges in these sophisticated tasks due to the inherent difficulties in modeling, annotation, and the evaluation associated with existing training paradigms [15]. Furthermore, the recently developed Llama-3 model has been trained on an extensive corpus

[†] Work done while Zhengwei Tao was interning at Alibaba Group.

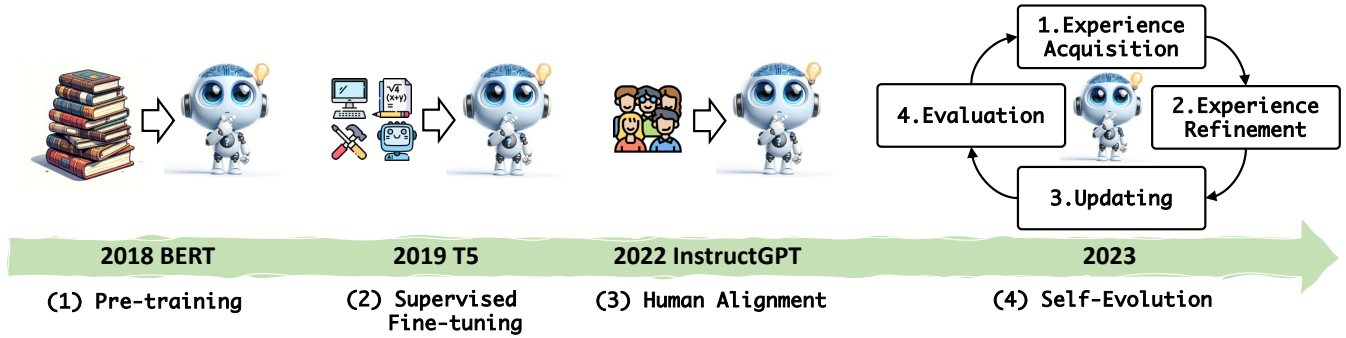


Fig. 1 Training paradigms shift of LLMs.

comprising 15 trillion tokens¹. It’s a monumental volume of data, suggesting that significantly scaling model performance by adding more real-world data could pose a limitation.

This has attracted interest in self-evolving mechanisms for LLMs, akin to the natural evolution of human intelligence and illustrated by AI developments in gaming, such as the transition from AlphaGo [101] to AlphaZero [102]. AlphaZero’s self-play method, requiring no labeled data, showcases a path forward for LLMs to surpass current limitations and achieve superhuman performance without intensive human supervision.

Drawing inspiration from the paradigm above, research on the self-evolution of LLMs has rapidly increased at different stages of model development, such as self-instruct [127], self-play [117], self-improving [52], and self-training [42]. Notably, DeepMind’s AMIE system [117] outperforms primary care physicians in diagnostic accuracy, and Microsoft’s WizardLM-2² exceeds the performance of the initial version of GPT-4. Both models are developed using self-evolutionary frameworks with autonomous learning capabilities and represent a potential LLM training paradigm shift. However, the relationships between these methods remain unclear, lacking systematic organization and analysis.

Therefore, we first comprehensively investigate the self-evolution processes in LLMs and establish a conceptual framework for their development. This self-evolution is characterized by an iterative cycle involving experience acquisition, experience refinement, updating, and evaluation, as shown in Figure 2. During the cycle, an LLM initially gains experiences through evolving new tasks and generating corresponding solutions, subsequently refining these experiences to obtain better supervision signals. After updating the model in-weight or in-context, the LLM is evaluated to measure progress and set new objectives.

The concept of self-evolution in LLMs has sparked considerable excitement across various research communities, promising a new era of models that can adapt, learn, and improve autonomously, akin to human evolution in response to changing environments and challenges. Self-evolving LLMs are not only able to transcend the limitations of current static, data-bound models but also mark a shift toward more dynamic, robust, and intelligent systems. This survey deepens understanding of the emerging field of self-evolving LLMs by providing a comprehensive overview through a structured conceptual framework. We trace the field’s evolution from the past to the latest cutting-edge methods and applications while examining existing challenges and outlining future research directions, paving the way for significant advances in developing self-evolution frameworks and next-generation models.

The survey is organized as follows: We first present the overview of self-evolution (§ 2), including background and conceptual framework. We summarize existing evolving abilities and domains of current methods (§ 3). Then, we provide in-depth analysis and discussion on the latest advancements in different phases of the self-evolution process, including experience acquisition (§ 4), experience refinement (§ 5), updating (§ 6), and evaluation (§ 7). Finally, we outline open problems and prospective future directions (§ 8).

2 Overview

In this section, we will first discuss the background of self-evolution and then introduce the proposed conceptual framework.

¹<https://huggingface.co/meta-llama/Meta-Llama-3-70B-Instruct>

²<https://wizardlm.github.io/WizardLM2/>

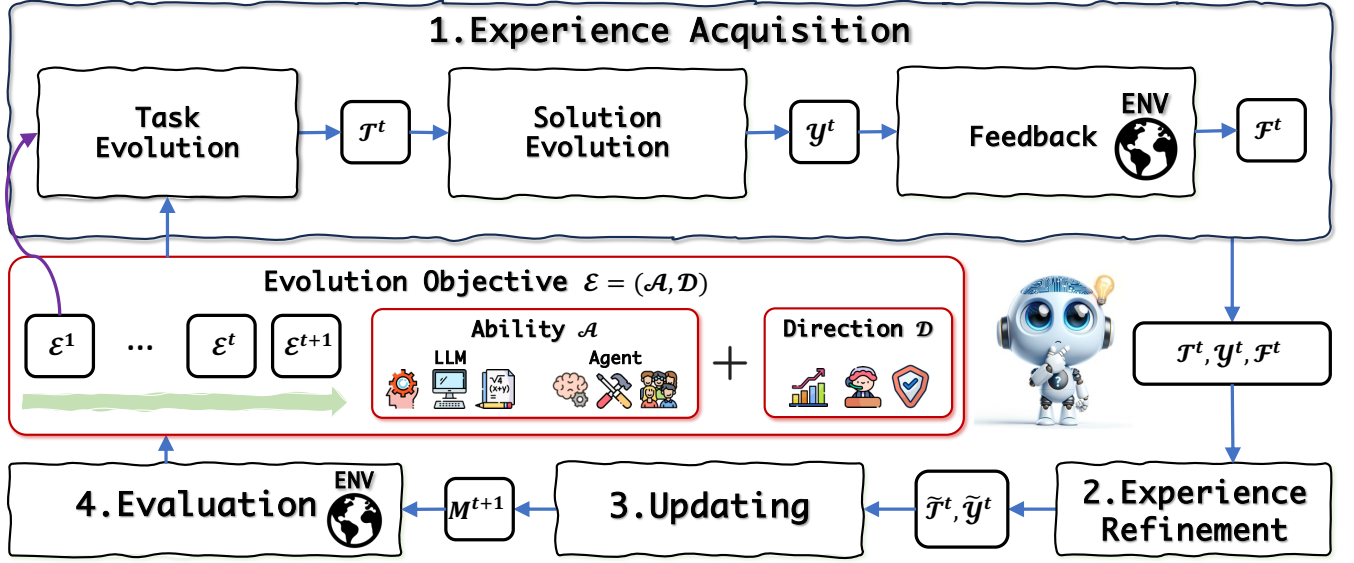


Fig. 2 Conceptual framework of self-evolution. For the t^{th} iteration: $\mathcal{E}^t$ is the evolution objective; $\mathcal{T}^t$ and $\mathcal{Y}^t$ denote the task and solution; $\mathcal{F}^t$ represents feedback; M^t is the current model. Refined experiences are marked as $\tilde{\mathcal{T}}^t$ and $\tilde{\mathcal{Y}}^t$, leading to the evolved model $\tilde{M}$. ENV is the environment. The whole self-evolution starts at $\mathcal{E}^1$.

2.1 Background

Self-Evolution in Artificial Intelligence. Artificial Intelligence represents an advanced form of intelligent agent, equipped with cognitive faculties and behaviors mirroring those of humans. The aspiration of AI developers lies in enabling AI to harness self-evolutionary capabilities, paralleling the experiential learning processes characteristic of human development. The concept of self-evolution in AI emerges from the broader fields of machine learning and evolutionary algorithms [7]. Initially influenced by the principles of natural evolution, such as selection, mutation, and reproduction, researchers have developed algorithms that simulate these processes to optimize solutions to complex problems. The landmark paper by Holland [47], which introduced the genetic algorithm, marks a foundational moment in the history of AI’s capability for self-evolution. Subsequent developments in neural networks and deep learning have furthered this capability, allowing AI systems to modify their own architectures and improve performance without human intervention [74].

Can Artificial Entities Evolve Themselves? Philosophically, the question of whether artificial entities can self-evolve touches on issues of autonomy, consciousness, and agency. While some philosophers argue that true self-evolution in AI would require some form of consciousness or self-awareness, others maintain that mechanical self-improvement through algorithms does not constitute genuine evolution [16]. This debate often references the works of thinkers like Dennett [28], who explore the cognitive processes under human consciousness and contrast them with artificial systems. Ultimately, the philosophical inquiry into AI’s capacity for self-evolution remains deeply intertwined with interpretations of what it means to ‘evolve’ and whether such processes can purely be algorithmic or must involve emergent consciousness [97].

2.2 Conceptual Framework

In the conceptual framework of self-evolution, we describe a dynamic, iterative process mirroring the human ability to acquire and refine skills and knowledge. This framework is encapsulated within Figure 2, emphasizing the cyclical nature of learning and improvement. Each iteration of the process focuses on a specific evolution goal, allowing the model to engage in relevant tasks, optimize its experiences, update its architecture, and evaluate its progress before moving to the next cycle.

Experience Acquisition

At the t^{th} iteration, the model identifies an evolution objective $\mathcal{E}^t$. Guided by this objective, the model embarks on new tasks $\mathcal{T}^t$, generating solutions $\mathcal{Y}^t$ and receiving feedback $\mathcal{F}^t$ from the environment, ENV. This stage culminates in the acquisition of new experiences $(\mathcal{T}^t, \mathcal{Y}^t, \mathcal{F}^t)$.

Experience Refinement

After experience acquisition, the model examines and refines these experiences. This involves discarding incorrect data and enhancing imperfect ones, resulting in refined outcomes $(\tilde{\mathcal{T}}^t, \tilde{\mathcal{Y}}^t)$.

Updating

Leveraging the refined experiences, the model undergoes an update process, integrating $(\tilde{\mathcal{T}}^t, \tilde{\mathcal{Y}}^t)$ into its framework. This ensures the model remains current and optimized.

Evaluation

The cycle concludes with an evaluation phase, where the model’s performance is assessed through an evaluation in external environment. The outcomes of this phase inform the objective $\mathcal{E}^{t+1}$, setting the stage for the subsequent iteration of self-evolution.

The conceptual framework outlines the self-evolution of LLMs, akin to human-like acquisition, refinement, and autonomous learning processes. We illustrate our taxonomy in Figure 3.

3 Evolution Objectives

Evolution objectives in self-evolving LLMs serve as predefined goals that autonomously guide their development and refinement. Much like humans set personal objectives based on needs and desires, these objectives are crucial as they determine how the model iteratively self-updates. They enable the LLM to autonomously learn from new data, optimize algorithms, and adapt to changing environments, effectively “feeling” its needs from feedback or self-assessment and setting its own goals to enhance functionality without human intervention.

We define an evolution objective as combining an evolving ability and an evolution direction. An evolving ability stands for an innate and detailed skill. The evolution direction is the aspect of the evolution objective aiming to improve. We formulate the evolution objective as follows:

$$\mathcal{E}^t = (\mathcal{A}^t, \mathcal{D}^t), \quad (1)$$

where $\mathcal{E}^t$ is the evolution objective, composed by evolving abilities $\mathcal{A}^t$ and evolution directions $\mathcal{D}^t$. Take “reasoning accuracy improving” as an example, “reasoning” is the evolving ability and “accuracy improving” is the evolution direction.

3.1 Evolving Abilities

In Table 1, we summarize and categorize the targeted evolving abilities in current self-evolution research into two groups: LLMs and LLM Agents.

3.1.1 LLMs

These are fundamental abilities underlying a broad spectrum of downstream tasks.

Instruction Following: The capability to follow instructions is essential for effectively applying language models. It allows these models to address specific user needs across different tasks and domains, aligning their responses within the given context [134].

Reasoning: LLMs can self-evolve to recognize statistical patterns, making logical connections and deductions based on the information. They evolve to perform better reasoning involving methodically dissecting problems in a logical sequence [27].

Math: LLMs enhance the intricate ability to solve mathematical problems covering arithmetic, math word, geometry, and automated theorem proving [2] towards self-evolution.

Coding: Methods improve the LLM coding abilities to generate more precise and robust programs [103, 150]. Furthermore, EvoCodeBench [62] provides an evolving benchmark that updates periodically to prevent data leakage.

Role-Play: It involves an agent understanding and acting out a particular role within a given context. This is crucial in scenarios where the model must fit into a social structure or follow a set of behaviors associated with a specific identity or function [77].

Others: Apart from the above fundamental evolution objectives, self-evolution can also achieve a wide range of NLP tasks [42, 59, 106, 152, 153].

METHOD	ACQUISITION			REFINEMENT $f^{\mathcal{R}}$	UPDATING $f^{\mathcal{U}}$	OBJECTIVE $\mathcal{E}$
	TASK $f^{\mathcal{T}}$	SOLUTION $f^{\mathcal{V}}$	FEEDBACK $f^{\mathcal{F}}$			
LARGE LANGUAGE MODELS						
Self-Align [108]	Knowledge-Based	Pos-G	-	Filtering	In-W	IF
SciGLM [152]	Knowledge-Based	-	-	-	In-W	Other
EvIT [110]	Knowledge-Based	-	-	-	In-W	Reasoning
MEEL [111]	Knowledge-Based	-	-	-	In-W	Reasoning
UltraChat [32]	Knowledge-Based	-	-	-	In-W	Role-Play
SOLID [6]	Knowledge-Based	Pos-S	-	Filtering	In-W	Role-Play
Ditto [77]	Knowledge-Based	Pos-S, Neg-P	-	-	In-W	Role-Play
MetaMath [146]	Knowledge-Free	Pos-R	-	-	In-W	Math
Self-Rewarding [148]	Knowledge-Free	-	Model	-	In-W	IF,Reasoning,Role-Play
Kun [158]	Knowledge-Free	-	-	Filtering	In-W	IF,Reasoning
PromptBreeder [36]	Knowledge-Free	-	-	-	In-C	Math, Reasoning
Ada-Instruct [27]	Knowledge-Free	-	-	-	In-W	Math, Reasoning, Code
Backtranslation [67]	Knowledge-Free	-	-	-	In-W	IF
DiverseEvol [131]	Selective	-	-	-	In-W	Code
Grath [21]	Selective	Neg-C	Model	-	In-W	Reasoning
REST ^{em} [103]	Selective	-	Model	Filtering	In-W	Math, Code
SOFT [122]	Selective	-	-	-	In-W	IF
LSX [106]	-	Pos-R	Model	Correcting	In-W	Other
LMSI [52]	-	Pos-R	-	Filtering	In-W	Math
TRAN [140]	-	Pos-G	-	-	In-C	Reasoning
MOT [66]	-	Pos-R, Pos-G	-	Filtering	In-C	Math, Reasoning
STaR [149]	-	Pos-R, Neg-C	Model	Correct	In-W	Reasoning
COTERRORSET [114]	-	Pos-R, Neg-C	-	-	In-W	Math, Reasoning
Self-Debugging [22]	-	Pos-I	Env	-	In-C	Code
SelfEvolve [56]	-	Pos-I	Env	-	In-C	Code
Reflexion [98]	-	Pos-I, Pos-G	-	-	In-C	Code, Reasoning
V-STaR [49]	-	Neg-C	Model	Filter	In-W	Math, Code
Self-Contrast [155]	-	Neg-C	Model	-	In-W	Reasoning
SALMON [107]	-	Neg-C	Model	-	In-W	IF,Reasoning,Role-Play
SPIN [24]	-	Neg-C	-	-	In-W	IF,Reasoning,Role-Play
RLCD [138]	-	Neg-P	Model	-	In-W	IF
DLMA [70]	-	Neg-P	Model	-	In-W	IF
SELF [76]	-	-	Model	Correct	In-W	IF, Math
LLM AGENTS						
AutoAct [90]	Knowledge-Based	Pos-I	Env	Filtering	In-W	Planning, Tool
KnowAgent [165]	Knowledge-Based	Pos-I, Pos-G	Env	Filtering	In-W	Embodied, Planning, Tool
RoboCat [13]	Knowledge-Free	Pos-I	Env	-	In-W	Embodied
STE [120]	Knowledge-Free	Pos-I, Neg-C	Env	Correct	In-W	Tool
IML [121]	-	Pos-R, Pos-G	-	-	In-C	Reasoning
SinViG Xu et al [135]	-	Pos-I	Env	Filtering	In-W	Embodied
ETO [105]	-	Pos-I, Neg-C	Env	Correct	In-W	Tool
A ³ T [141]	-	Pos-I, Neg-C	Env	Correct	In-W	Tool
Debates [112]	-	Pos-S	-	-	In-W	Communication
SOTOPIA- π [125]	-	Pos-S, Pos-G	Env	-	In-W	Communication
Self-Talk [118]	-	Pos-S, Pos-G	Model	Filtering	In-W	Communication
MemGPT [84]	-	Pos-G	Env	Filtering	In-C	Communication
MemoryBank [161]	-	Pos-G	Env	Filtering	In-C	Communication
ProAgent [151]	-	Pos-G	Env	-	In-C	Embodied
Agent-Pro [154]	-	Pos-G	Env	-	In-C	Planning
AesopAgent [123]	-	Pos-G	Env	-	In-C	Planning
ICE [89]	-	Pos-G	Env	-	In-C	Planning
TiM [72]	-	Pos-G	-	-	In-C	Communication
Werewolf [136]	-	Pos-G	-	-	In-C	Planning

Table 1 Overview of self-evolution methods, detailing approaches across evolutionary stages. Key: Pos (Positive), Neg (Negative), R (Rationale-based), I (Interactive), S (Self-play), G (Grounded), C (Contrastive), P (Perturbative), Env (Environment), In-W (In-Weight), In-C (In-Context), IF (Instruction-Following). We use colors to highlight evolution directions: **Adaptation to Feedback** in green, **Expansion of Knowledge Base** in blue, and **Safety, Ethics, and Bias Reduction** in brown. Performance enhancement remains in black.

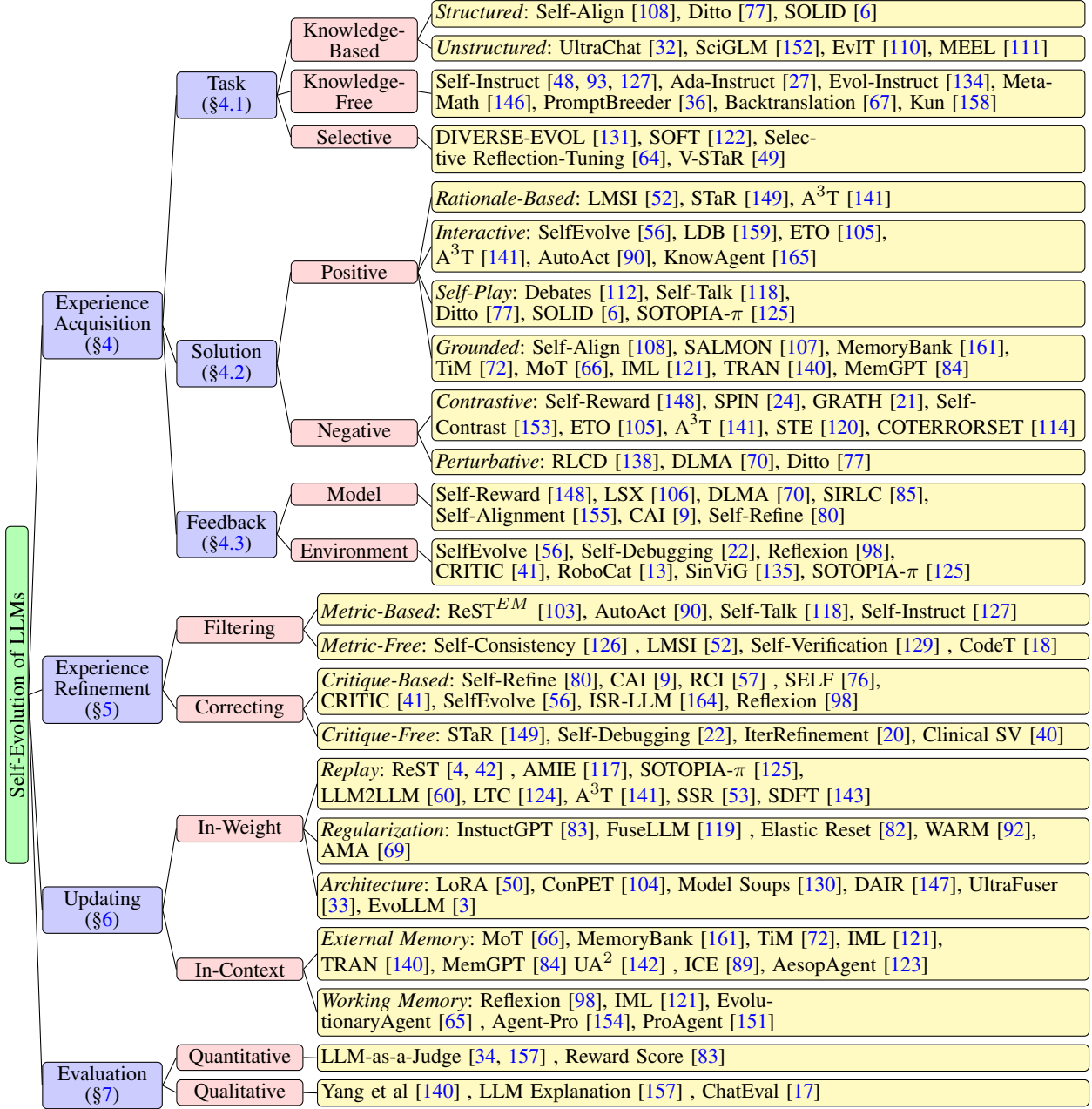


Fig. 3 Taxonomy of self-evolving large language models.

3.1.2 LLM-based Agents

The abilities discussed here are characteristic of advanced artificial agents used for task-solving or simulations in digital or physical world. These capabilities mirror human cognitive functions, allowing these agents to perform complex tasks and interact effectively in dynamic environments.

Planning: It involves the ability to strategize and prepare for future actions or goals. An agent with this skill can analyze the current state, predict the outcomes of potential actions, and create a sequence of steps to achieve a specific objective [90].

Tool Use: This is the capacity to employ objects or instruments in the environment to perform tasks, manipulate surroundings, or solve problems [165].

Embodied Control: It refers to an agent’s ability to manage and coordinate its physical form within an environment. This encompasses locomotion, dexterity, and the manipulation of objects [13].

Communication: It is the skill to convey information and understand messages from other agents or humans. Agents with advanced communication abilities can participate in dialogue, collaborate with others, and adjust their behaviour based on the communication received [118].

3.2 Evolution Directions

Examples of evolution directions include but are not limited to:

Improving Performance: The goal is to continuously enhance the model’s understanding and generation across various languages and abilities. For instance, a model initially trained for question answering and chitchat can autonomously extend its proficiency and develop abilities like diagnostic dialogue [117], social skills [125], and role-playing [77].

Adaptation to Feedback: This involves improving model responses based on feedback to better align with preferences or adapt to environments [108, 138].

Expansion of Knowledge Base: The aim is to continuously update the model’s knowledge base with the latest information and trends. For example, a model might automatically integrate new scientific research into its responses [132].

Safety, Ethic and Reducing Bias: The goal is to identify and mitigate biases in the model’s responses, ensuring fairness and safety. One effective strategy is to incorporate guidelines, such as constitutions or specific rules, to identify inappropriate or biased responses and correct them through model updates [9, 78].

4 Experience Acquisition

Exploration and exploitation [44] are fundamental strategies for learning in humans and LLMs. Among that, exploration involves seeking new experiences to achieve objectives and is analogous to the initial phase of LLM self-evolution, known as experience acquisition. This process is crucial for self-evolution, enabling the model to autonomously tackle core challenges such as adapting to new tasks, overcoming knowledge limitations, and enhancing solution effectiveness. Furthermore, experience is a holistic construct, encompassing not only the tasks encountered [31] but also the solutions developed to address these tasks [96], and the feedback [12] received as a result of task performance.

Inspired by that, we divide experience acquisition into three parts: task evolution, solution evolution, and obtaining feedback. In task evolution, LLMs curate and evolve new tasks aligning with evolution objectives. For solution evolution, LLMs develop and implement strategies to complete these tasks. Finally, LLMs may optionally collect feedback from interacting with the environment for further improvements.

4.1 Task Evolution

To gain new experience, the model first evolves new tasks according to the evolution objective $\mathcal{E}^t$ in the current iteration. Task evolution is the crucial step in the engine that starts the entire evolution process. Formally, we denote the task evolution as:

$$\mathcal{T}^t = f^{\mathcal{T}}(\mathcal{E}^t, M^t), \quad (2)$$

where $f^{\mathcal{T}}$ is the task evolution function. $\mathcal{E}^t$, M^t , and $\mathcal{T}^t$ denote the evolution objective, the model, and the evolved task at iteration t , respectively. We summarize and categorize existing studies on the task evolution method $f^{\mathcal{T}}$ into three groups: *Knowledge-Based*, *Knowledge-Free*, and *Selective*. We detail each type in the following parts and show the concepts in Figure 4.

Knowledge-Based

The objective $\mathcal{E}^t$ may associate with external knowledge to evolve where the knowledge is not inherently comprised in the current LLMs. Explicitly sourcing from knowledge enriches the relevance between tasks and evolution objectives. It also ensures the validity of relevant facts in the tasks. We delve into the Knowledge-Based methods seeking to evolve new tasks of the evolving objective assisted by external information.

The first kind of knowledge is structured. Structured knowledge is dense in information and well-organized. Self-Align [108] guides task generation by covering 20 topics, such as scientific and legal expertise, to ensure diversity. Apart from topic knowledge, DITTO [77] includes character knowledge from Wikidata and Wikipedia. The knowledge comprises attributes, profiles, and concise character details for role-play conversations. SOLID [6] generates structured entity knowledge as conversation starters.

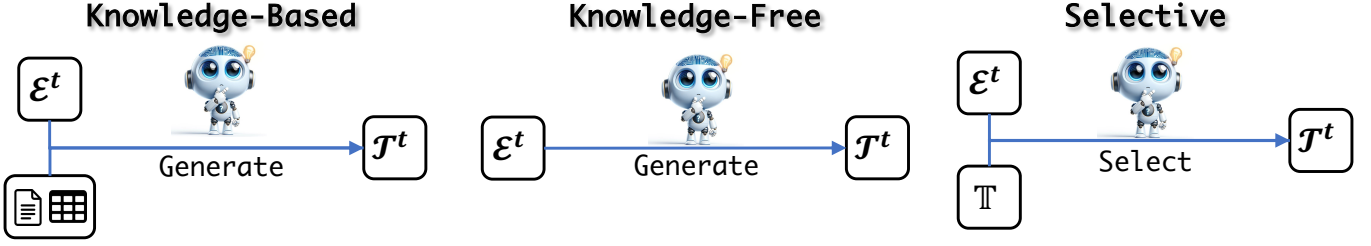


Fig. 4 Task evolution. $\mathcal{E}^t$ and $\mathcal{T}^t$ are the evolving objective and task of t^{th} iteration. $\mathbb{T}$ is the set of all tasks to be selected. The first two are generative methods that differ based on their respective use of knowledge. The third method, in contrast, employs a discriminative approach to select what to learn.

The second group consists of tasks evolving from an unstructured context. Unstructured context is easy to obtain but is sparse in knowledge. UltraChat [32] gathers unstructured knowledge of 20 types of text materials to construct questions or instructions. SciGLM [152] derives questions from the text of diversified science subjects, which covers rich scientific knowledge. EvIT [110] derives event reasoning tasks based on large-scale unstructured events mined from the unsupervised corpus. Similarly, MEEL [111] evolves multi-modal events in both image and text to construct the tasks for MM event reasoning.

Knowledge-Free

Unlike previous methods that require extensive human effort to gather external knowledge, Knowledge-Free approaches operate independently using the evolving object $\mathcal{E}^t$ and the model itself. These efficient methods can generate more diversified tasks without additional knowledge restrictions.

First, the LLMs can prompt themselves to generate new tasks according to $\mathcal{E}^t$. Self-Instruct [48, 93, 127] is a typical methodology of Knowledge-Free task evolution. These methods self-generate a variety of new task instructions based on evolution objectives. Ada-Instruct [27] further proposes an adaptive task instruction generation strategy that fine-tunes open-source LLMs to generate lengthy and complex task instructions for code completion and mathematical reasoning.

Second, extending and boosting original tasks increases the quality of instructions. WizardLM [134] proposes Evol-Instruct that evolves instruction following tasks with in-depth and in-breadth evolving and further expands it in code generation [79]. MetaMath [146] rewrites the question in multiple ways, including rephrasing, self-verification, and FOBAR. It evolves a new MetaMathQA dataset for fine-tuning LLMs to improve mathematical task-solving. Promptbreeder [36] evolves seed tasks via mutation prompts. It further evolves mutation prompts via the hyper mutation prompts to increase the task diversity.

Third, deriving tasks from plain text is another way. Backtranslation [67] extracts self-contained segments in unlabelled data and regards it as the answers to tasks. Similarly, Kun [158] presents a task self-evolving algorithm utilizing instruction harnessed from unlabelled data towards back-translation.

Selective

Instead of task generation, we may start with large-scale existing tasks. At each iteration, LLMs can select tasks that exhibit the highest relevance to the current evolving objective $\mathcal{E}^t$ without additional generation. This approach obviates the intricate curation of new tasks, streamlining the evolution process [19, 63, 162].

A simple task selecting method is to randomly sample tasks from the task pool like REST [42], REST^{em} [103], and GRATH [21] do. Rather than random selection, DIVERSE-EVOL [131] introduces a data sampling technique where the model selects new data points based on their distinctiveness in the embedding space, ensuring diversity enhancement in the chosen subset. SOFT [122] then splits the initial training set. Each iteration selects one chunk of the split set as the evolving task.

Li et al [64] propose Selective Reflection-Tuning and select a subset of tasks via a novel metric calculating to what extent the answer is related to the question. V-STaR [49] selects the correct solutions in the previous iteration and adds their task instructions to the task set of the next iteration.

4.2 Solution Evolution

After obtaining evolved tasks, LLMs solve the tasks to acquire the corresponding solution. The most common strategy is to generate the solution directly according to the task formulation [42, 103, 148, 149, 158]. However, this straightforward approach might reach solutions irrelevant to the evolution objective, leading to suboptimal evolution [45]. Therefore, solution evolution

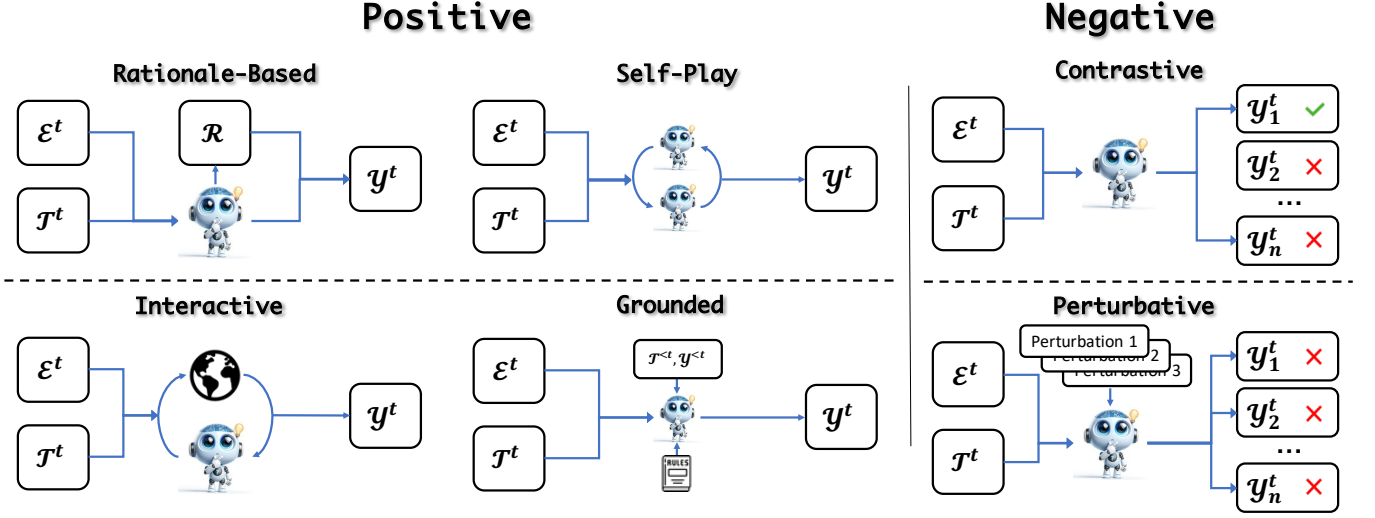


Fig. 5 Solution evolution. $\mathcal{E}^t$, $\mathcal{T}^t$, and y^t are the evolving objective, task, and solution of t^{th} iteration. $\mathcal{R}$ is the rational thought.

uses different strategies to solve tasks and enhance LLM capabilities by ensuring that solutions are not just generated but are also relevant and informative. In this section, we comprehensively survey these strategies and illustrate them in Figure 5. We first formulate the solution-evolution as follows:

$$y^t = f^{\mathcal{Y}}(\mathcal{T}^t, \mathcal{E}^t, M^t), \quad (3)$$

where $f^{\mathcal{Y}}$ is the model’s strategy to approach the evolution objective.

We then categorize these methods into positive and negative according to the correctness of the solutions. The positive methods introduce various approaches to acquire correct and desirable solutions. On the contrary, negative methods elicit and collect undesired solutions, including unfaithful or mis-align model behaviors, which are then used for preference alignment. We elaborate on the details of each type in the following sections.

4.2.1 Positive

Current studies explore diverse methods beyond vanilla inference for positive solutions to obtain correct solutions aligned with evolution objectives. We categorize the task-solving process into four types: Rationale-Based, Interactive, Self-Play, and Grounded.

Rationale-Based

The model incorporates rationale explanations towards approaching the evolving objective when solving the tasks and can self-evolve by utilizing such rationales. These methods enable models to explicitly acknowledge the evolution objective and complete this task in that direction [10, 128, 144, 145].

Huang et al [52] proposes a method where an LLM self-evolves using “high-confidence” rationale-augmented answers generated for unlabeled questions. Similarly, STaR [149] generates rationale when solving the task. If the answer is wrong, it further corrects the rationale and the answer. Then, it uses the answer and rationale as experiences to fine-tune the model. Similarly, LSX [106] proposes the novel paradigm to generate an explanation of the answer, incorporating an iterative loop between a learner module performing a base task and a critic module that assesses the quality of explanations given by the learner. Song et al [105], Yang et al [141] obtain rationales in the ReAct [144] style when solving the tasks. The rationales are further engaged in training the agents in the following step.

Interactive

Models can interact with the environment to enhance the evolution process. These methods can obtain environmental feedback that is valuable for guiding self-evolution directions.

SelfEvolve and LDB [56, 159] improve code generation ability via self-evolution. They allow the model to generate code and acquire feedback via running the code on the interpreter. As another environment, Song et al [105], Yang et al [141] interact in embodied scenarios and acquire feedback. They learn to take proper actions based on their current state. For agent abilities, AutoAct [90] introduces self-planning from scratch, focusing on an intrinsic self-learning process. In this process, agents enhance their abilities through recursive planning iterations with environment feedback. Following AutoAct, [165] further enhances agent training by integrating self-evolution and an external action knowledge base. This approach guides action generation and boosts planning ability through environment-driven corrective feedback loops.

Self-Play

It’s the situation where a model learns to evolve by playing against copies of itself. Self-play is a powerful evolving method because it enables systems to communicate with themselves to get feedback in a closed loop. It’s especially effective in environments where the model can simulate various sides of the roles, like multi-player games [101, 102]. Compared with interactive methods, self-play is an effective strategy to obtain feedback without an environment.

Taubenfeld et al [112] investigate the systematic biases in simulations of debates by LLMs. On the contrary to debating, Ulmer et al [118] engage LLMs in conversations following generated principles. Another kind of conversation via role-playing. Lu et al [77] proposes self-simulated role-play conversation. The process involves instructing the LLM with character profiles and aligning its responses to maintain consistency with the character’s knowledge and style. Similarly, Askari et al [6] propose SOLID to generate large-scale intent-aware role-play dialogues. This self-playing aspect harnesses the expansive knowledge of LLMs to construct information-rich exchanges that streamline the dialog generation process. Wang et al [125] introduces a novel approach whereby each LLM follows a role and communicates with each other to achieve their goals.

Grounded

To reach the evolving objective and reduce exploration space, models can be grounded on existing rules [108] and previous experiences for further explicit guidance when solving the tasks.

LLMs can generate desirable solutions more effectively by being grounded on pre-defined rules and principles. For instance, Self-Align [108] generated self-evolved questions with principle-driven constraints to guide the task-solving process. SALMON [107] design a set of combined principles that requires the model to follow when solving the task. Self-Talk [118] ensures the LLMs generate a workflow-aligned conversation based on preset agent characters. They generate the workflow in advance based on GPT-4.

Besides pre-defined rules, grounding on previous experiences can improve the solutions. MemoryBank [161] and TiM [72] answer current questions by incorporating previous question-answer records. Rather than previous solution histories, MoT [66], IML [121], and TRAN [140] incorporate induced rules from the histories to answer new questions. MemGPT [84] combines these merits and retrieves previous questions, solutions, induced events, and user portrait knowledge.

4.2.2 Negative

In addition to acquiring positive solutions, recent research illustrates that LLMs can benefit from negative ones for self-improvement [140]. This strategy is analogous to trial and error in human behavior when learning skills. This section summarises typical methods of gaining negative solutions to assist in self-evolution.

Contrastive

A widely used group of methods is to collect multiple solutions for a task and then contrast the positive and negative ones to get improvements.

For instance, Self-Reward [148] generates responses, predicts corresponding rewards by the model itself, and constructs preference pairs based on rewards. SPIN [24] uses the results generated by the current model as negatives and the ground truth of SFT as positives for iterative self-training. Similarly, GRATH [21] generates both correct and incorrect answers. It then trains the model by comparing these two answers. Self-Contrast [153] contrasts the differences and summarizes these discrepancies into a checklist that could be used to re-examine and eliminate discrepancies. In ETO [105], the model interacts with the embodied environment to complete tasks and optimizes from the failure solutions. A³T [141] improves ETO by adding rationale after each action for solving tasks. STE [120] implements trial and error where the model solves the tasks with unfamiliar tools. It learns by analyzing failed attempts to improve problem-solving strategies in future tasks. More recently, COTERRORSET [114] obtains incorrect solutions generated by PALM-2 and proposes mistake tuning, which requires the model to void making mistakes.

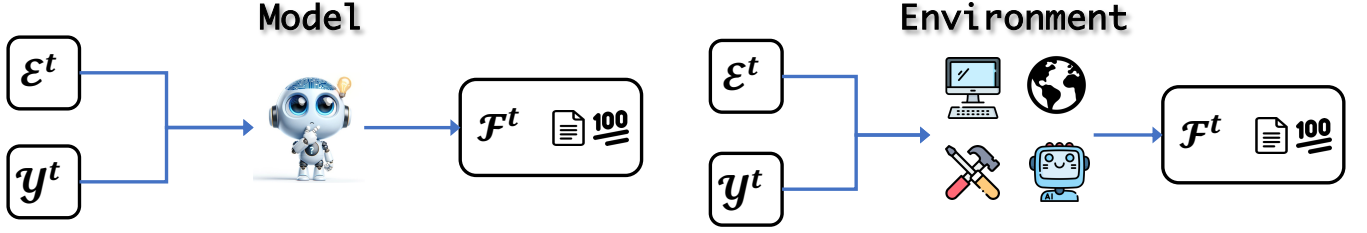


Fig. 6 Types of feedback. $\mathcal{E}^t$ and $\mathcal{Y}^t$ are the evolving objective and task solutions of t^{th} iteration.

Perturbative

Compared to *Contrastive*, *Perturbative* methods seek to add perturbations to obtain negative solutions intentionally. Models can later learn to avoid generating these negative answers. Adding perturbations to obtain negative solutions is more controllable than contrastive methods.

Some methods add perturbation to generate harmful solutions [70, 138]. Given a task, RLCD [138] curates both positive and negative instructions and generates positive and negative solutions. DLMA [70] gathers both positive and negative instructional prompts and subsequently produces corresponding positive and negative solutions.

Rather than harmful perturbation, incorporating negative context is another way. Ditto [77] adds negative persona characters to generate incorrect conversations. The model then learns from the negative conversations to evolve persona dialogue ability.

4.3 Feedback

As humans learn skills, feedback plays a critical role in demonstrating the correctness of the solutions. This key information enables humans to reflect and then update their skills. Akin to this process, LLMs should obtain feedback during or after the task solution in the cycle of self-evolution. We formalize the process as follows:

$$\mathcal{F}^t = f^{\mathcal{F}}(\mathcal{T}^t, \mathcal{Y}^t, \mathcal{E}^t, \mathcal{M}^t; \text{ENV}), \quad (4)$$

where $f^{\mathcal{F}}$ is the method to acquire feedback.

In this part, we summarize two types of feedback. Model feedback refers to gathering the critique or score rated by the LLMs themselves. Besides, Environment denotes the feedback received directly from the external environment. We illustrate these concepts in Figure 6.

4.3.1 Model

Current studies demonstrate that LLMs can play well as a critic [157]. In the cycle of self-evolution, the model judges itself to acquire the feedback of the solutions.

One type of feedback is a score that indicates correctness. Self-Reward [148], LSX Stammer et al [106], and DLMA [70] rate their own solutions and output the scores via LLM-as-a-Judge prompting. Similar to that, SIRLC [85] utilizes self-evaluation results of LLM as the reward for further reinforcement learning. Self-Alignment [155] leverages the self-evaluation capability of an LLM to generate confidence scores on the factual accuracy of its outputs.

Another type provides a textual description, offering multi-dimensional information. To alter the distribution of the responses via supervised learning, CAI [9] asks the model to critique its response according to a principle in the constitution. In contrast to supervised learning and reinforcement learning approaches, Self-Refine [80] allows the model to generate natural language feedback on its own output in a few-shot manner.

4.3.2 Environment

Another form of feedback comes from the environment, common in tasks where solutions can be directly evaluated. This feedback is precise and elaborate and can provide sufficient information for model updating. They may be derived from code interpreter [22, 56, 98], tool execution [41, 90], the embodied environment [13, 135, 164], and other LLMs or agents [112, 118, 125].

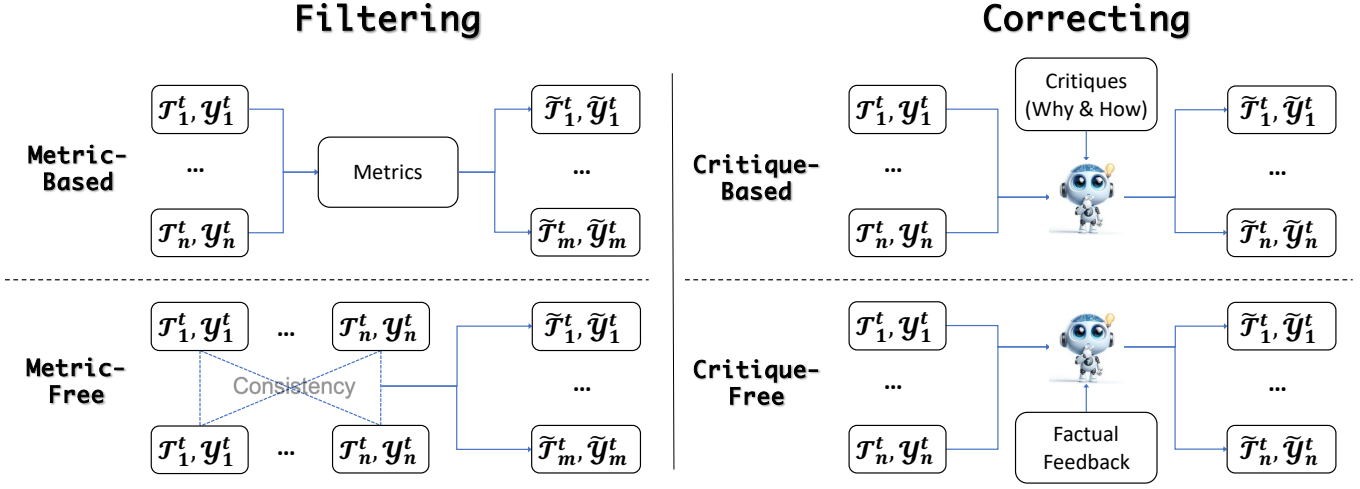


Fig. 7 Experience refinement. $\tilde{\mathcal{T}}^t$ and $\tilde{\mathcal{Y}}^t$ are the task and solution after refinement, respectively. Filtering reduces the number of experiences from n to m .

For code generation, Self-Debugging Chen et al [22] utilizes execution results on test cases as part of feedback while Self-Evolve [56] receives the error message from the interpreter. Similarly, Reflexion [98] also obtains the run-time feedback from the code interpreter. It then further reflects to generate thoughts. This run-time feedback contains the trace-back information that can point out the key information for improved code generation.

Recently, methods endow tool-using ability to LLMs and agents. Executing tools leading to feedback in return [41, 90, 105, 120, 141].

RoboCat [13] and SinViG [135] act in the robotic embodied environment. This type of feedback is precise and strong to guide self-evolution.

Communication feedback is common and effective in LLM-based multi-agent systems. Agents can correct and support each other, enabling co-evaluation [112, 118, 125].

5 Experience Refinement

After experience acquisition and before updating in self-evolution, LLMs may improve the quality and reliability of their outputs through experience refinement. It helps LLMs adapt to new information and contexts without relying on external resources, leading to more reliable and effective assistance in dynamic environments. This process is formulated as follows:

$$\tilde{\mathcal{T}}^t, \tilde{\mathcal{Y}}^t = f^{\mathcal{R}}(\mathcal{T}^t, \mathcal{Y}^t, \mathcal{F}^t, \mathcal{E}^t, \mathcal{M}^t), \quad (5)$$

where $f^{\mathcal{R}}$ is the methods of experience refinement, $\tilde{\mathcal{T}}^t, \tilde{\mathcal{Y}}^t$ are the refined tasks and solutions. We classify the methods into two categories: filtering and correcting.

5.1 Filtering

Refinement in self-evolution involves two primary filtering strategies: *Metric-Based* and *Metric-Free*. The former uses external metrics to assess and filter outputs, while the latter does not rely on these metrics. This ensures that only the most reliable and high-quality data is utilized for further updating.

5.1.1 Metric-Based

By relying on feedback and pre-defined criteria, metric-based filtering improves the quality of the outputs [90, 103, 118, 127], ensuring the progressive enhancement of LLM capabilities through each iteration of refinement.

For example, ReST^{EM} [103] incorporates a reward function to filter the dataset sampled from the current policy. The function provides binary rewards based on the correctness of the generated samples rather than a learned reward model trained

on human preferences in ReST [42]. AutoAct [90] leverages F1-score and accuracy as rewards for synthetic trajectories and collects trajectories with exactly correct answers for further training. Self-Talk [118] measures the number of completed sub-goals to filter the generated dialogues, ensuring that only high-quality data is used for training. To encourage diversity of the source instructions, Self-Instruct [127] automatically filters low-quality or repeated instructions using ROUGE-L similarity and heuristics before adding them to the task pool.

The filtering criteria or metrics are crucial for maintaining the quality and reliability of the generated outputs, thereby ensuring the continuous improvement of the model’s capability.

5.1.2 Metric-Free

Some methods seek filtering strategies beyond external metrics, making the process more flexible and adaptable. *Metric-Free* filtering typically involves sampling outputs and evaluating them based on internal consistency measures or other model-inherent criteria [18, 52, 129]. The filtering in Self-Consistency [126] is based on the consistency of the final answer across multiple generated reasoning paths, with higher agreement indicating higher reliability. LMSI [52] utilizes CoT prompting plus self-consistency for generating high-confidence self-training data.

Designing internal consistency measures that accurately reflect output quality can be challenging. Self-Verification [129] allows the model to select the candidate answer with the highest interpretable verification score, calculated by assessing the consistency between the predicted and original condition values. For the code generation task, CodeT [18] considers both the consistency of the outputs against the generated test cases and the agreement of the outputs with other code samples.

These methods emphasize the language model’s ability to self-assess and filter its outputs based on internal agreement, showcasing a significant step forward in self-evolution without the direct intervention of external metrics.

5.2 Correcting

Recent advancements emphasize the importance of iterative self-correction in LLMs, allowing for the refinement of experiences. This section divides the methods employed into two categories: *Critique-Based* and *Critique-Free* correction. Critiques often serve as strong hints that include the rationale behind perceived errors or suboptimal outputs, guiding the model towards improved iterations.

5.2.1 Critique-Based

These methods rely on additional judging processes to draw the critiques of the experiences. Then, the experiences are refined based on the critiques. By leveraging either self-generated [9, 76, 80, 98] or environment-interaction generated critiques [41, 56, 164], the model benefits from detailed feedback for nuanced correction.

LLMs have demonstrated their ability to identify errors in their outputs. Self-Refine [80] introduces an iterative process in which the model refines its initial outputs conditioned on actionable self-feedback without additional training. To evolve from the correction, CAI [9] generates critiques and revisions of its outputs in the supervised learning phase, significantly improving the initial model. Applied to an agent automating computer tasks, RCI [57] improves its previous outputs based on the critique finding errors in the outputs.

Since weaker models may struggle significantly with the self-critique process, several approaches enable models to correct the outputs using critiques provided by external tools. CRITIC [41] allows LLMs to revise the output based on the critiques obtained during interaction with tools in general domains. SelfEvolve [56] prompts an LLM to refine the answer code based on the error information thrown by the interpreter. ISR-LLM [164] helps the LLM planner find a revised action plan by using a validator in an iterative self-refinement process.

The primary advantage of this method lies in its ability to process and react to detailed feedback, potentially leading to more targeted and nuanced corrections.

5.2.2 Critique-Free

Contrary to critique-based, critique-free methods correct the experiences directly leveraging objective information [20, 22, 40, 149]. These methods offer the advantage of independence from nuanced feedback that critiques provide, allowing for corrections that adhere strictly to factual accuracy or specific guidelines without the potential bias introduced by critiques.

One group of critique-free methods modifies the experiences on the signal of whether the task was correctly resolved. Self-Taught Reasoner (STaR) [149] proposes a technique that iteratively generates rationales to answer questions. If the answers are

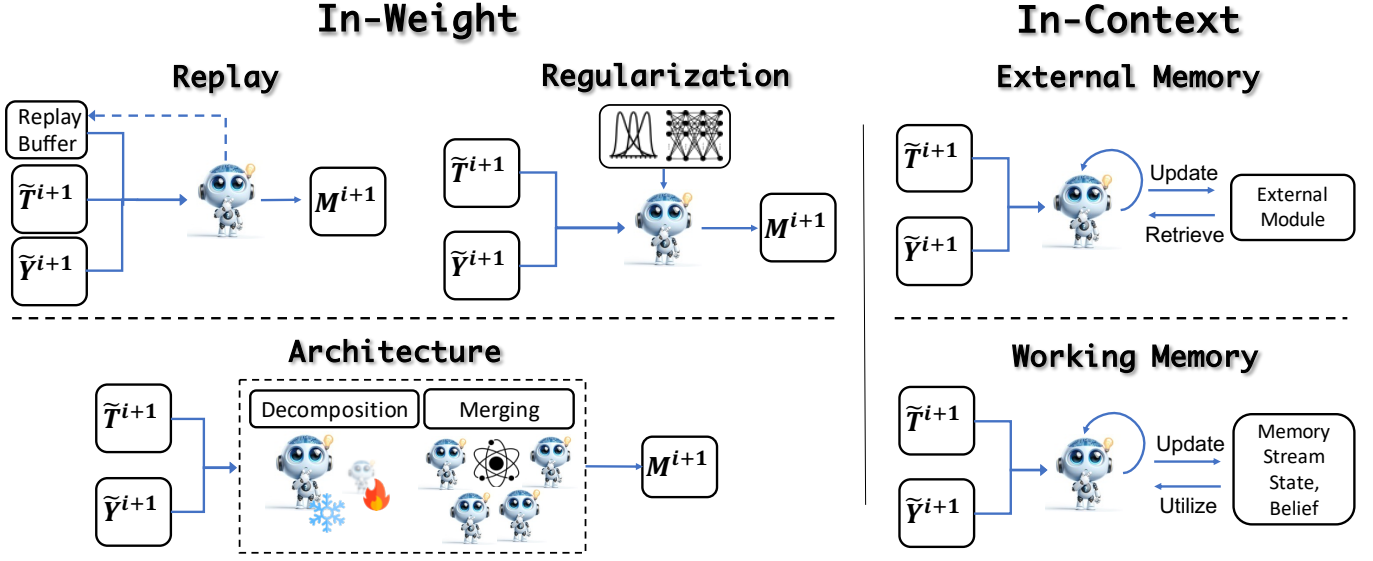


Fig. 8 The illustration of *Updating* methods, including in-weight and in-context updating. The terms $\tilde{T}^t$ and $\tilde{Y}^t$ represent refined experiences, each containing task and corresponding solutions, respectively. M^t denotes the updated model.

incorrect, the model is prompted again with the correct answer to generate a more informed rationale. Self-Debug [22] enables the model to perform debugging steps by investigating execution results from unit tests and explaining the code on its own.

Different from depending on the task-solving signal, other information produced during the solving process can be leveraged. IterRefinement [20] relies on a series of refined prompts that encourage the model to reconsider and improve upon its previous outputs without any direct critique. For information extraction tasks, Clinical SV [40] grounds each element in evidence from the input and prunes inaccurate elements using supplied evidence.

These critique-free approaches simplify the correction mechanism, allowing for easier implementation and faster adjustments.

6 Updating

After experience refinement, we enter the crucial updating phase that leverages the refined experiences to improve model performance. We formulate updating as follows:

$$M^{t+1} = f^{\mathcal{U}}(\tilde{T}^t, \tilde{Y}^t, \mathcal{E}^t, M^t), \quad (6)$$

where $f^{\mathcal{U}}$ is the updating functions. These update methods keep the model effective by adapting to new experiences and continuously improving performance in changing environments and during iterative training.

We divide these approaches into in-weight learning, which involves updates to model weights, and in-context learning, which involves updates to external or working memory.

6.1 In-Weight

Classical training paradigms in updating LLMs in weight encompass continuous pretraining [14, 93], supervised fine-tuning [75], and preference alignment [83, 115]. However, in the iterative training process of self-evolving, the core challenge lies in **achieving overall improvement** and **preventing catastrophic forgetting**, which entails refining or acquiring new capabilities while preserving original skills. Solutions to this challenge can be categorized into three main strategies: replay-based, regularization-based, and architecture-based methods.

6.1.1 Replay-based

Replay-based methods reintroduce previous data to retain old knowledge. One is experience replay, which mixes the original and new training data to update LLMs [60, 93, 124, 141, 156]. For example, Reinforced Self-Training (ReST) [4, 42] method iteratively updates large language models by mixing seed training data with filtered new outputs generated by the model itself. AMIE [117] utilizes a self-play simulated learning environment for iterative improvement and mixes generated dialogues with supervised fine-tuning data through inner and outer self-play loops. SOTOPIA- π [125] leverages behavior cloning from the expert model and self-generated social interaction trajectory to reinforce positive behaviors.

Another is generative replay, which adopts the self-generated synthesized data as knowledge to mitigate catastrophic forgetting. For instance, Self-Synthesized Rehearsal (SSR) [53] generates synthetic training instances for rehearsal, enabling the model to preserve its ability without relying on real data from previous training stages. Self-Distillation Fine-Tuning (SDFT) [143] generates a distilled dataset from the model itself to bridge the distribution gap between task datasets and the LLM’s original distribution to mitigate catastrophic forgetting.

6.1.2 Regularization-based

Regularization-based methods constrain the model’s updates to prevent significant deviations from original behaviors, exemplified by function- and weight-based regularization. Function-based regularization focuses on modifying the loss function that a model optimizes during training [87, 160]. For example, InstuctGPT [83] employs a per-token KL-divergence penalty from the output probabilities of the initial policy model π_{SFT} on the updated policy model π_{RL} . FuseLLM [119] employs a technique akin to knowledge distillation [46], leveraging the generated probability distributions from source LLMs to transfer the collective knowledge to the target LLM.

Weight-based regularization [58] directly targets the model’s weights during training. Techniques such as Elastic Reset [82] counters alignment drift in RLHF by periodically resetting the online model to an exponentially moving average of its previous states. Furthermore, Ramé et al [92] introduced WARM, which combines multiple reward models through weight averaging to address reward hacking and misalignment. Moreover, AMA [69] adaptively average model weights to optimize the trade-off between reward maximization and forgetting mitigation.

6.1.3 Architecture-based

Architecture-based methods explicitly utilize extra parameters or models for updating, including decomposition- and merging-based approaches. Decomposition-based methods separate large neural network parameters into general and task-specific components and only update the task-specific parameters to mitigate forgetting. LoRA [29, 50] inject trainable low-rank matrices to significantly reduce the number of trainable parameters while maintaining or improving model performance across various tasks. This paradigm is later adopted by GPT4tools [139], OpenAGI [39] and Dromedary [108]. Dynamic ConPET [104] combines pre-selection and prediction with task-specific LoRA modules to prevent forgetting, ensuring scalable and effective adaptation of LLMs to new tasks.

Merging-based methods, on the other hand, involve combining multiple models or layers to achieve general improvements, including but not limited to merging multiple generic and specialized model weights into a single model [55, 130, 137, 147], through mixture-of-expert approach [33] or even layer-wise merging and up-scaling such as EvoLLM [3].

6.2 In-Context

In addition to directly updating model parameters, another approach is to leverage the in-context capabilities of LLMs to learn from experiences, thereby enabling fast adaptive updates without expensive training costs. The methods could be divided into updating external and working memory.

6.2.1 External Memory

This approach utilizes an external module to collect, update, and retrieve past experiences and knowledge, enabling models to access a rich pool of insights and achieve better results without updating model parameters. The external memory mechanism is common in AI Agent systems [89, 123, 136]. This section provides a detailed overview of the latest methods for updating external memory, emphasizing the aspects of memory *Content* and *Updating Operations*, and summarized in Table 2.

METHOD	CONTENT	OPERATION
MoT [66]	Experience	Insert
TRAN [140]	Rationale	Insert, Reflect
MemoryBank [161]	Experience, Rationale	Insert, Reflect, Forget
MemGPT [84]	Experience	Insert, Forget
TiM [72]	Rationale	Insert
IML [121]	Rationale	Insert, Reflect
ICE [89]	Rationale	Insert, Reflect
AesopAgent [123]	Experience, Rationale	Insert, Reflect

Table 2 Content and operations of updating external memory.

Content: External memory mainly stores two types of content: past experiences and reflected rationale, each serving distinct purposes. For instance, past experience provides valuable historical context, serving as a guiding force toward achieving improved outcomes. MoT [66] archives filtered question-answer pairs to construct a beneficial memory repository. Additionally, the FIFO Queue mechanism in MemGPT [84] maintains a rolling history of messages, encapsulating interactions between agents and users, system notifications, and inputs and outputs of function calls.

On the other hand, reflected rationales offer condensed explanations, such as rules, that support decision-making. For instance, TRAN [140] archives rules inferred from experiences alongside information on mistakes to mitigate future errors. Correspondingly, TiM [72] preserves inductive reasoning, defined as text elucidating the relationships between entities. Moreover, IML [121] and ICE [89] store comprehensive notes and rules derived from a series of trajectories, demonstrating the broad spectrum of content types that memory systems can accommodate.

MemoryBank [161] and AesopAgent [123] reflect on experience and rationale, save them as external memory, and retrieve them when needed to achieve better performance.

Updating Operation: We categorize the operations to the memory into Insert, Reflect, and Forget. The most common operation is insert, methods insert text content into the memory for storage [66, 72, 84, 121, 140, 161]. Another operation is reflection, which is to think and summarize previous experiences to conceptualize rules and knowledge for future use [89, 121, 140, 161]. Last, due to the limited storage of memory, forgetting content is crucial to keeping memory efficient and the content valid. MemGPT [84] adopts the FIFO queue to forget the contents. MemoryBank [161] establishes a forgetting curve on the insert time of each item.

6.2.2 Working Memory

The methods use past experience to evolve the capabilities of agents by updating internal memory streams, states, or beliefs, known as working memory, often in the form of verbal cues. Reflexion [98] introduces verbal reinforcement learning for decision-making improvement without conventional model updates. Similarly, IML [121] enables LLM-based agents to autonomously learn and adapt to their environment by summarizing, refining, and updating knowledge based on past experience directly in working memory.

EvolutionaryAgent [65] aligns agents with dynamically changing social norms through evolution and selection principles, leveraging environmental feedback for self-evolution. Agent-Pro [154] employs policy-level reflection and optimization, allowing agents to adapt their behavior and beliefs in interactive scenarios based on past outcomes. Lastly, ProAgent [151] enhances cooperation in multi-agent systems by dynamically interpreting teammates’ intentions and adapting behavior.

These collective works demonstrate the importance of integrating past experiences and knowledge into the agents’ memory stream to refine their state or beliefs for improved performance and adaptability across various tasks and environments.

7 Evaluation

Much like the human learning process, it is essential to ascertain whether the present level of ability is adequate and meets the application requirements through evaluation. Furthermore, it is from these evaluations that one can identify the direction for future learning. However, how to accurately assess the performance of an evolved model and provide directions for future improvements is a crucial yet underexplored research area. For a given evolved model M^t , we conceptualize the evaluation process as follows:

$$\mathcal{E}^{t+1}, \mathcal{S}^t = f^{\mathcal{E}}(M^t, \mathcal{E}^t, \text{ENV}), \quad (7)$$

where f^E represents the evaluation function that measures the performance score (S^{t+1}) of the current model and provide evolving goal ($\mathcal{E}^{t+1}$) for the next iteration. Evaluation function f^E can be categorized into quantitative and qualitative approaches, each providing valuable insights into model performance and areas for improvement.

7.1 Quantitative Evaluation

This method focuses on providing measurable metrics to reliably assess LLM performance, such as automatic [68, 86] and human evaluation. However, traditional automatic metrics struggle to accurately evaluate increasingly complex tasks, and human assessment is not an ideal option for autonomous self-evolution. Recent trends use LLMs as human proxy for automatic evaluators, offering cost-effective and scalable solutions for evaluations.

For example, reward model score has been widely used to measure model or task performances [98] and select the best checkpoint [83]. LLM-as-a-judge [157] using LLMs to evaluate LLMs, employing methods like pairwise comparison, single answer grading, and reference-guided grading. It shows that LLMs can closely match human judgment, enabling efficient large-scale evaluations.

7.2 Qualitative Evaluation

Qualitative evaluation involves case studies and analysis to derive insights, offering evolving guidance for subsequent iterations. Initiatives such as LLM-as-a-judge [157] provide the reasoning behind its assessments; ChatEval [17] explores the strengths and weaknesses of model outputs through debate mechanisms. Furthermore, TRAN [140] leverages past errors to formulate rules that enhance future LLM performances. Nonetheless, compared with instance-level critic or reflection, qualitative evaluation at the task- or model-level still needs comprehensive investigation.

8 Open Problems

8.1 Objectives: Diversity and Hierarchy

Section 3 summarizes existing evolution objectives and their coverage. Nonetheless, these highlighted objectives can only satisfy a small fraction of the vast human needs. The extensive application of LLMs across various tasks and industries highlights unresolved challenges in establishing self-evolution frameworks for evolving objectives that can comprehensively address a broader spectrum of real-world tasks [35].

Furthermore, the concept of evolving objectives entails a potential hierarchical structure; for instance, UltraTool [54] and T-Eval [23] categorize the capability of tool usage into various sub-dimensions. Exploring evolutionary objectives into manageable sub-goals and pursuing them individually emerges as a viable strategy.

Overall, a clear and urgent need exists to develop self-evolution frameworks that effectively address diversified and hierarchical objectives.

8.2 Level of Autonomy: From Low to High

Self-evolution in large models is emerging, yet lacks clear definitions for its autonomous levels. We categorize self-evolution into three tiers: low, medium, and high-level autonomy

Low-level

In this level, the user predefined the evolving object $\mathcal{E}$ and it remains unchanged. The user needs to design the evolving pipeline, namely all modules $f^\bullet$, on its own. Then, the model completes the self-evolution process based on the designed framework. We denote this level of self-evolution in the following formula:

$$\tilde{M} = \text{Evol}^L(M, \mathcal{E}, f^\bullet, \text{ENV}), \quad (8)$$

where M denotes the model to be evolved. $\tilde{M}$ is the evolving output. ENV is the environment. Most of the current works lie at this level.

Medium-level

In this level, the user only sets the evolving object $\mathcal{E}$ and keeps it unchanged. The user doesn't need to design the specific modules $f^\bullet$ in the framework. The model can construct each module $f^\bullet$ independently for self-evolution. This level denotes as follows:

$$\tilde{M} = \text{Evol}^M(M, \mathcal{E}, \text{ENV}), \quad (9)$$

High-level

In the final level, the model diagnoses its deficiency and constructs the self-evolution methods to improve itself. This is the ultimate purpose of self-evolution. The user model sets its own evolving object $\mathcal{E}$ according to the evaluation f^E output. The evolving objective would change during the iteration. Besides, the model designs the specific modules $f^\bullet$ in the framework. We represent this level as:

$$\tilde{M} = \text{Evol}^H(M, \text{ENV}), \quad (10)$$

As discussed in the previous open problem (§ 8.1), there are a large of unfulfilled objectives. However, most of the existing self-evolution frameworks are at the Low-level which requires specifically designed modules [77, 90, 148]. These frameworks are objective-dependent and rely on large human efforts to develop. Exhausting all objectives are not deployment-efficient which brings about the urgent need to develop medium and high levels self-evolution frameworks. At the medium level, it doesn't require expert efforts to design specific modules. LLMs can self-evolve according to targeted objectives. Then at the high level, LLMs can investigate their current deficiencies and evolve in a targeted manner. In all, developing highly autonomous self-evolution frameworks remains an open problem.

8.3 Experience Acquisition and Refinement: From Empirical to Theoretical

Suppose we have addressed the previous two challenges and developed promising self-evolution frameworks, but the exploration of self-evolution LLMs still lacks solid theoretical grounding. This idea posits that LLMs can self-improve or correct their outputs, with or without feedback from the environment. However, the mechanisms behind it remain unclear. Studies show mixed results: Huang et al [51] observed self-corrective behavior in models with over 22 billion parameters, while Ganguli et al [38] finds LLMs struggle to self-correct reasoning errors without external feedback.

A related challenge is the use of self-generated data for learning. Critics argue this approach could reduce linguistic diversity [43] and lead to "model collapse," where models fail to capture complex, long-tailed data distributions [100]. Furthermore, Alemohammad et al [5] reveal that generative models trained on their synthetic outputs progressively lose output quality and diversity. Fu et al [37] extend this by theoretically analyzing the impact of self-consuming training loops on model performance, emphasizing the importance of balancing synthetic and real data to mitigate error accumulation.

Recent studies [103, 141] also show that current methods struggle to improve after more than three rounds of self-evolution. One hypothesized reason is that the self-critic of LLM has not co-evolved with the evolving objective, but more experimental and theoretical support is still needed. These findings highlight a pressing need for more theoretical exploration in self-evolving LLMs. Addressing these concerns is crucial for advancing the field and ensuring that models can effectively learn and improve over time.

8.4 Updating: Stability-Plasticity Dilemma

The stability-plasticity dilemma represents a crucial yet unresolved challenge that is essential for iterative self-evolution. This dilemma reflects the difficulty of balancing the need to retain previously learned information (stability) while adapting to new data or tasks (plasticity). Existing LLMs either overlook this issue or adopt conventional methods that may be ineffective. While training models from scratch could mitigate the problem of catastrophic forgetting, it is highly inefficient, particularly as model parameters increase exponentially and autonomous learning capabilities advance. Finding a balance between acquiring new skills and preserving existing knowledge is crucial for achieving effective and efficient self-evolution, leading to overall improvement.

8.5 Evaluation: Systematic and Evolving

To effectively assess LLMs, a dynamic, comprehensive benchmark is crucial. This becomes even more pivotal as we progress towards Artificial General Intelligence (AGI). Traditional static benchmarks risk obsolescence due to LLMs' evolving nature

and potential access to test data through interacting with environments, such as search engines, undermining their reliability. A dynamic benchmark, like Sotopia [163], proposes a solution by creating an LLM-based environment tailored for evaluating the social intelligence of LLMs, thereby avoiding the limitations posed by static benchmarks.

8.6 Safety and Superalignment

The advancement of LLMs opens the possibility for AI systems to achieve or even surpass expert-level capabilities in both supportive and autonomous decision-making. For safety, ensuring these LLMs align with human values and preferences is crucial, particularly to mitigate inherent biases that can impact areas such as political debates, as highlighted by Taubenfeld et al [112]. OpenAI’s initiative, Superalignment [61], aims to align a superintelligence by developing scalable training methods, validating models for alignment, and stress-testing the alignment process through scalable oversight [94], robustness [88], automated interpretability [11], and adversarial testing. Although challenges remain, Superalignment marks an initial attempt to develop a self-evolving LLM that closely aligns with human ethics and values in a scalable way.

9 Conclusion

The evolution of LLMs towards self-evolution paradigms represents a transformative shift in artificial intelligence akin to the human learning process. It is promising to overcome the limitations of current models that rely heavily on human annotation and teacher models. This survey presents a comprehensive framework for understanding and developing self-evolving LLMs, structured around iterative cycles of experience acquisition, refinement, updating, and evaluation. By detailing advancements and categorizing the evolution objectives within this framework, we offer a thorough overview of current methods and highlight the potential for LLMs to adapt, learn, and improve autonomously. We also identify existing challenges and propose directions for future research, aiming to accelerate the progress toward more dynamic, intelligent, and efficient models. This work deepens the understanding of self-evolving LLMs. It paves the way for significant advancements in AI, marking a step towards achieving superintelligent systems capable of surpassing human performance in complex real-world tasks.

References

- [1] Achiam J, Adler S, Agarwal S, et al (2023) Gpt-4 technical report. arXiv preprint arXiv:230308774
- [2] Ahn J, Verma R, Lou R, et al (2024) Large language models for mathematical reasoning: Progresses and challenges. In: Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: Student Research Workshop, URL <https://aclanthology.org/2024.eacl-srw.17>
- [3] Akiba T, Shing M, Tang Y, et al (2024) Evolutionary optimization of model merging recipes. arXiv preprint arXiv:240313187
- [4] Aksitov R, Miryoosefi S, Li Z, et al (2023) Rest meets react: Self-improvement for multi-step reasoning llm agent. arXiv preprint arXiv:231210003
- [5] Alemohammad S, Casco-Rodriguez J, Luzi L, et al (2024) Self-consuming generative models go MAD. In: The Twelfth International Conference on Learning Representations
- [6] Askari A, Petcu R, Meng C, et al (2024) Self-seeding and multi-intent self-instructing llms for generating intent-aware information-seeking dialogs. arXiv preprint arXiv:240211633
- [7] Bäck T, Schwefel HP (1993) An overview of evolutionary algorithms for parameter optimization. *Evolutionary computation* 1(1):1–23
- [8] Bai J, Bai S, Chu Y, et al (2023) Qwen technical report. arXiv preprint arXiv:230916609
- [9] Bai Y, Kadavath S, Kundu S, et al (2022) Constitutional ai: Harmlessness from ai feedback. arXiv preprint arXiv:221208073

- [10] Besta M, Blach N, Kubicek A, et al (2024) Graph of thoughts: Solving elaborate problems with large language models. In: Proceedings of the AAAI Conference on Artificial Intelligence, pp 17682–17690
- [11] Bills S, Cammarata N, Mossing D, et al (2023) Language models can explain neurons in language models. URL <https://openaipublic.blob.core.windows.net/neuron-explainer/paper/index.html>(Date accessed: 1405 2023)
- [12] Boud D, Keogh R, Walker D (2013) Reflection: Turning experience into learning. Routledge
- [13] Bousmalis K, Vezzani G, Rao D, et al (2023) Robocat: A self-improving generalist agent for robotic manipulation. Transactions on Machine Learning Research
- [14] Brown T, Mann B, Ryder N, et al (2020) Language models are few-shot learners. Advances in neural information processing systems 33:1877–1901
- [15] Burns C, Izmailov P, Kirchner JH, et al (2023) Weak-to-strong generalization: Eliciting strong capabilities with weak supervision. arXiv preprint arXiv:231209390
- [16] Chalmers DJ (1997) The conscious mind: In search of a fundamental theory. Oxford Paperbacks
- [17] Chan CM, Chen W, Su Y, et al (2024) Chateval: Towards better LLM-based evaluators through multi-agent debate. In: The Twelfth International Conference on Learning Representations
- [18] Chen B, Zhang F, Nguyen A, et al (2023) Codet: Code generation with generated tests. In: The Eleventh International Conference on Learning Representations
- [19] Chen L, Li S, Yan J, et al (2023) Alpapasus: Training a better alpaca with fewer data. arXiv preprint arXiv:230708701
- [20] Chen P, Guo Z, Haddow B, et al (2023) Iterative translation refinement with large language models. arXiv preprint arXiv:230603856
- [21] Chen W, Li B (2024) Grath: Gradual self-truthifying for large language models. arXiv preprint arXiv:240112292
- [22] Chen X, Lin M, Schaerli N, et al (2023) Teaching large language models to self-debug. In: The 61st Annual Meeting Of The Association For Computational Linguistics
- [23] Chen Z, Du W, Zhang W, et al (2023) T-eval: Evaluating the tool utilization capability step by step. arXiv preprint arXiv:231214033
- [24] Chen Z, Deng Y, Yuan H, et al (2024) Self-play fine-tuning converts weak language models to strong language models. arXiv preprint arXiv:240101335
- [25] Chung HW, Hou L, Longpre S, et al (2022) Scaling instruction-finetuned language models. arXiv preprint arXiv:221011416
- [26] Collins KM, Jiang AQ, Frieder S, et al (2023) Evaluating language models for mathematics through interactions. arXiv preprint arXiv:230601694
- [27] Cui W, Wang Q (2023) Ada-instruct: Adapting instruction generators for complex reasoning. arXiv preprint arXiv:231004484
- [28] Dennett DC (1993) Consciousness explained. Penguin uk
- [29] Dettmers T, Pagnoni A, Holtzman A, et al (2024) Qlora: Efficient finetuning of quantized llms. Advances in Neural Information Processing Systems 36

- [30] Devlin J, Chang MW, Lee K, et al (2018) Bert: Pre-training of deep bidirectional transformers for language understanding. arXiv preprint arXiv:1810.04805
- [31] Dewey J (1938) Experience and education: Kappa delta pi. International Honor Society in Education
- [32] Ding N, Chen Y, Xu B, et al (2023) Enhancing chat language models by scaling high-quality instructional conversations. arXiv preprint arXiv:2305.14233
- [33] Ding N, Chen Y, Cui G, et al (2024) Mastering text, code and math simultaneously via fusing highly specialized language models. arXiv preprint arXiv:2403.08281
- [34] Dubois Y, Li CX, Taori R, et al (2024) AlpacaFarm: A simulation framework for methods that learn from human feedback. Advances in Neural Information Processing Systems 36
- [35] Eloundou T, Manning S, Mishkin P, et al (2023) GPTs are GPTs: An early look at the labor market impact potential of large language models. arXiv preprint arXiv:2303.10130
- [36] Fernando C, Banarse D, Michalewski H, et al (2023) PromptBreeder: Self-referential self-improvement via prompt evolution. arXiv preprint arXiv:2309.16797
- [37] Fu S, Zhang S, Wang Y, et al (2024) Towards theoretical understandings of self-consuming generative models. arXiv preprint arXiv:2402.11778
- [38] Ganguli D, Askell A, Schiefer N, et al (2023) The capacity for moral self-correction in large language models. arXiv preprint arXiv:2302.07459
- [39] Ge Y, Hua W, Mei K, et al (2024) Openagi: When LLM meets domain experts. Advances in Neural Information Processing Systems 36
- [40] Gero Z, Singh C, Cheng H, et al (2023) Self-verification improves few-shot clinical information extraction. In: ICML 3rd Workshop on Interpretable Machine Learning in Healthcare (IMLH), URL <https://openreview.net/forum?id=SBbJICrgIS>
- [41] Gou Z, Shao Z, Gong Y, et al (2024) CRITIC: Large language models can self-correct with tool-interactive critiquing. In: The Twelfth International Conference on Learning Representations
- [42] Gulcehre C, Paine TL, Srinivasan S, et al (2023) Reinforced self-training (rest) for language modeling. arXiv preprint arXiv:2308.08998
- [43] Guo Y, Shang G, Vazirgiannis M, et al (2023) The curious decline of linguistic diversity: Training language models on synthetic text. arXiv preprint arXiv:2311.09807
- [44] Gupta AK, Smith KG, Shalley CE (2006) The interplay between exploration and exploitation. Academy of management journal 49(4):693–706
- [45] Hare J (2019) Dealing with sparse rewards in reinforcement learning. arXiv preprint arXiv:1910.09281
- [46] Hinton G, Vinyals O, Dean J (2015) Distilling the knowledge in a neural network. arXiv preprint arXiv:1503.02531
- [47] Holland JH (1992) Adaptation in natural and artificial systems: an introductory analysis with applications to biology, control, and artificial intelligence. MIT press
- [48] Honovich O, Scialom T, Levy O, et al (2023) Unnatural instructions: Tuning language models with (almost) no human labor. In: Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pp 14409–14428

- [49] Hosseini A, Yuan X, Malkin N, et al (2024) V-star: Training verifiers for self-taught reasoners. arXiv preprint arXiv:240206457
- [50] Hu EJ, yelong shen, Wallis P, et al (2022) LoRA: Low-rank adaptation of large language models. In: International Conference on Learning Representations, URL <https://openreview.net/forum?id=nZeVKeeFYf9>
- [51] Huang J, Chen X, Mishra S, et al (2023) Large language models cannot self-correct reasoning yet. In: The Twelfth International Conference on Learning Representations
- [52] Huang J, Gu SS, Hou L, et al (2023) Large language models can self-improve. In: Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, URL <https://aclanthology.org/2023.emnlp-main.67>
- [53] Huang J, Cui L, Wang A, et al (2024) Mitigating catastrophic forgetting in large language models with self-synthesized rehearsal. arXiv preprint arXiv:240301244
- [54] Huang S, Zhong W, Lu J, et al (2024) Planning, creation, usage: Benchmarking llms for comprehensive tool utilization in real-world complex scenarios. arXiv preprint arXiv:240117167
- [55] Ilharco G, Ribeiro MT, Wortsman M, et al (2023) Editing models with task arithmetic. In: The Eleventh International Conference on Learning Representations
- [56] Jiang S, Wang Y, Wang Y (2023) Selfevolve: A code evolution framework via large language models. arXiv preprint arXiv:230602907
- [57] Kim G, Baldi P, McAleer S (2024) Language models can solve computer tasks. Advances in Neural Information Processing Systems
- [58] Kirkpatrick J, Pascanu R, Rabinowitz N, et al (2017) Overcoming catastrophic forgetting in neural networks. Proceedings of the national academy of sciences 114(13):3521–3526
- [59] Koa KJ, Ma Y, Ng R, et al (2024) Learning to generate explainable stock predictions using self-reflective large language models. arXiv preprint arXiv:240203659
- [60] Lee N, Wattanawong T, Kim S, et al (2024) Llm2llm: Boosting llms with novel iterative data enhancement. arXiv preprint arXiv:240315042
- [61] Leike J, Sutskever I (2023) Introducing superalignment. URL <https://openai.com/blog/introducing-superalignment>, accessed: 2024-04-01
- [62] Li J, Li G, Zhang X, et al (2024) Evocodebench: An evolving code generation benchmark aligned with real-world code repositories. arXiv preprint arXiv:240400599
- [63] Li M, Zhang Y, Li Z, et al (2023) From quantity to quality: Boosting llm performance with self-guided data selection for instruction tuning. arXiv preprint arXiv:230812032
- [64] Li M, Chen L, Chen J, et al (2024) Selective reflection-tuning: Student-selected data recycling for llm instruction-tuning. arXiv preprint arXiv:240210110
- [65] Li S, Sun T, Qiu X (2024) Agent alignment in evolving social norms. arXiv preprint arXiv:240104620
- [66] Li X, Qiu X (2023) Mot: Memory-of-thought enables chatgpt to self-improve. In: Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, pp 6354–6374
- [67] Li X, Yu P, Zhou C, et al (2024) Self-alignment with instruction backtranslation. In: The Twelfth International Conference on Learning Representations, URL <https://openreview.net/forum?id=1oijHJBRsT>

- [68] Lin CY (2004) Rouge: A package for automatic evaluation of summaries. In: Text summarization branches out, pp 74–81
- [69] Lin Y, Lin H, Xiong W, et al (2024) Mitigating the alignment tax of rlhf. In: arXiv, [2309.06256](https://arxiv.org/abs/2309.06256)
- [70] Liu A, Bai H, Lu Z, et al (2024) Direct large language model alignment through self-rewarding contrastive prompt distillation. arXiv preprint arXiv:2402.11907
- [71] Liu J, Xia CS, Wang Y, et al (2024) Is your code generated by chatgpt really correct? rigorous evaluation of large language models for code generation. Advances in Neural Information Processing Systems 36
- [72] Liu L, Yang X, Shen Y, et al (2023) Think-in-memory: Recalling and post-thinking enable llms with long-term memory. arXiv preprint arXiv:2311.08719
- [73] Liu X, Yu H, Zhang H, et al (2024) Agentbench: Evaluating LLMs as agents. In: The Twelfth International Conference on Learning Representations, URL <https://openreview.net/forum?id=zAdUB0aCTQ>
- [74] Liu Y, Sun Y, Xue B, et al (2021) A survey on evolutionary neural architecture search. IEEE transactions on neural networks and learning systems 34(2):550–570
- [75] Longpre S, Hou L, Vu T, et al (2023) The flan collection: Designing data and methods for effective instruction tuning. In: Proceedings of the 40th International Conference on Machine Learning, pp 22631–22648, URL <https://proceedings.mlr.press/v202/longpre23a.html>
- [76] Lu J, Zhong W, Huang W, et al (2023) Self: Language-driven self-evolution for large language model. arXiv preprint arXiv:2310.00533
- [77] Lu K, Yu B, Zhou C, et al (2024) Large language models are superpositions of all characters: Attaining arbitrary role-play via self-alignment. arXiv preprint arXiv:2401.12474
- [78] Lu X, Yu B, Lu Y, et al (2024) Sofa: Shielded on-the-fly alignment via priority rule following. arXiv preprint arXiv:2402.17358
- [79] Luo Z, Xu C, Zhao P, et al (2024) Wizardcoder: Empowering code large language models with evol-instruct. In: The Twelfth International Conference on Learning Representations, URL <https://openreview.net/forum?id=UnUwSIgK5W>
- [80] Madaan A, Tandon N, Gupta P, et al (2024) Self-refine: Iterative refinement with self-feedback. Advances in Neural Information Processing Systems
- [81] Miret S, Krishnan N (2024) Are llms ready for real-world materials discovery? arXiv preprint arXiv:2402.05200
- [82] Noukhovitch M, Lavoie S, Strub F, et al (2024) Language model alignment with elastic reset. Advances in Neural Information Processing Systems 36
- [83] Ouyang L, Wu J, Jiang X, et al (2022) Training language models to follow instructions with human feedback. Advances in neural information processing systems
- [84] Packer C, Fang V, Patil SG, et al (2023) Memgpt: Towards llms as operating systems. arXiv preprint arXiv:2310.08560
- [85] Pang JC, Wang P, Li K, et al (2024) Language model self-improvement by reinforcement learning contemplation. In: The Twelfth International Conference on Learning Representations, URL <https://openreview.net/forum?id=38E4yUbrgr>
- [86] Papineni K, Roukos S, Ward T, et al (2002) Bleu: a method for automatic evaluation of machine translation. In: Proceedings of the 40th annual meeting of the Association for Computational Linguistics, pp 311–318

- [87] Peng K, Ding L, Zhong Q, et al (2023) Token-level self-evolution training for sequence-to-sequence learning. In: Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers), pp 841–850
- [88] Perez E, Huang S, Song F, et al (2022) Red teaming language models with language models. In: Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing, pp 3419–3448
- [89] Qian C, Liang S, Qin Y, et al (2024) Investigate-consolidate-exploit: A general strategy for inter-task agent self-evolution. arXiv preprint arXiv:240113996
- [90] Qiao S, Zhang N, Fang R, et al (2024) Autoact: Automatic agent learning from scratch via self-planning. arXiv preprint arXiv:240105268
- [91] Raffel C, Shazeer N, Roberts A, et al (2020) Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of machine learning research* 21(140):1–67
- [92] Ramé A, Vieillard N, Hussenot L, et al (2024) Warm: On the benefits of weight averaged reward models. arXiv preprint arXiv:240112187
- [93] Roziere B, Gehring J, Gloeckle F, et al (2023) Code llama: Open foundation models for code. arXiv preprint arXiv:230812950
- [94] Saunders W, Yeh C, Wu J, et al (2022) Self-critiquing models for assisting human evaluators. arXiv preprint arXiv:220605802
- [95] Schoenegger P, Park PS, Karger E, et al (2024) Ai-augmented predictions: Llm assistants improve human forecasting accuracy. arXiv preprint arXiv:240207862
- [96] Schön DA (2017) *The reflective practitioner: How professionals think in action*. Routledge
- [97] Searle JR (1986) *Minds, brains and science*. Harvard university press
- [98] Shinn N, Labash B, Gopinath A (2023) Reflexion: an autonomous agent with dynamic memory and self-reflection. arXiv preprint arXiv:230311366
- [99] Shinn N, Cassano F, Gopinath A, et al (2024) Reflexion: Language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems* 36
- [100] Shumailov I, Shumaylov Z, Zhao Y, et al (2023) The curse of recursion: Training on generated data makes models forget. arXiv preprint arXiv:230517493
- [101] Silver D, Huang A, Maddison CJ, et al (2016) Mastering the game of go with deep neural networks and tree search. *nature* 529(7587):484–489
- [102] Silver D, Hubert T, Schrittwieser J, et al (2017) Mastering chess and shogi by self-play with a general reinforcement learning algorithm. arXiv preprint arXiv:171201815
- [103] Singh A, Co-Reyes JD, Agarwal R, et al (2024) Beyond human data: Scaling self-training for problem-solving with language models. *Transactions on Machine Learning Research* URL <https://openreview.net/forum?id=INAYungGFK>, expert Certification
- [104] Song C, Han X, Zeng Z, et al (2023) Conpet: Continual parameter-efficient tuning for large language models. arXiv preprint arXiv:230914763

- [105] Song Y, Yin D, Yue X, et al (2024) Trial and error: Exploration-based trajectory optimization for llm agents. arXiv preprint arXiv:240302502
- [106] Stammer W, Friedrich F, Steinmann D, et al (2023) Learning by self-explaining. arXiv preprint arXiv:230908395
- [107] Sun Z, Shen Y, Zhang H, et al (2024) SALMON: Self-alignment with instructable reward models. In: The Twelfth International Conference on Learning Representations, URL <https://openreview.net/forum?id=xJbsmB8UMx>
- [108] Sun Z, Shen Y, Zhou Q, et al (2024) Principle-driven self-alignment of language models from scratch with minimal human supervision. Advances in Neural Information Processing Systems 36
- [109] Tan Y, Min D, Li Y, et al (2023) Evaluation of chatgpt as a question answering system for answering complex questions. arXiv preprint arXiv:230307992
- [110] Tao Z, Chen X, Jin Z, et al (2024) Evit: Event-oriented instruction tuning for event reasoning. [2404.11978](#)
- [111] Tao Z, Jin Z, Huang J, et al (2024) Meel: Multi-modal event evolution learning. [2404.10429](#)
- [112] Taubenfeld A, Dover Y, Reichart R, et al (2024) Systematic biases in llm simulations of debates. arXiv preprint arXiv:240204049
- [113] Team G, Anil R, Borgeaud S, et al (2023) Gemini: a family of highly capable multimodal models. arXiv preprint arXiv:231211805
- [114] Tong Y, Li D, Wang S, et al (2024) Can llms learn from previous mistakes? investigating llms' errors to boost for reasoning. arXiv preprint arXiv:240320046
- [115] Touvron H, Martin L, Stone K, et al (2023) Llama 2: Open foundation and fine-tuned chat models. arXiv preprint arXiv:230709288
- [116] Touvron H, Martin L, Stone K, et al (2023) Llama 2: Open foundation and fine-tuned chat models. arXiv preprint arXiv:230709288
- [117] Tu T, Palepu A, Schaekermann M, et al (2024) Towards conversational diagnostic ai. arXiv preprint arXiv:240105654
- [118] Ulmer D, Mansimov E, Lin K, et al (2024) Bootstrapping llm-based task-oriented dialogue agents via self-talk. arXiv preprint arXiv:240105033
- [119] Wan F, Huang X, Cai D, et al (2024) Knowledge fusion of large language models. In: The Twelfth International Conference on Learning Representations, URL <https://openreview.net/forum?id=jiDsk12qcz>
- [120] Wang B, Fang H, Eisner J, et al (2024) Llms in the imaginary: Tool learning through simulated trial and error. arXiv preprint arXiv:240304746
- [121] Wang B, Sun T, Yan H, et al (2024) In-memory learning: A declarative learning framework for large language models. arXiv preprint arXiv:240302757
- [122] Wang H, Ma G, Meng Z, et al (2024) Step-on-feet tuning: Scaling self-alignment of llms via bootstrapping. arXiv preprint arXiv:240207610
- [123] Wang J, Du Z, Zhao Y, et al (2024) Aesopagent: Agent-driven evolutionary system on story-to-video production. arXiv preprint arXiv:240307952
- [124] Wang K, Lu Y, Santacrose M, et al (2023) Adapting llm agents through communication. arXiv preprint arXiv:231001444

- [125] Wang R, Yu H, Zhang W, et al (2024) Sotopia- π : Interactive learning of socially intelligent language agents. arXiv preprint arXiv:240308715
- [126] Wang X, Wei J, Schuurmans D, et al (2023) Self-consistency improves chain of thought reasoning in language models. In: The Eleventh International Conference on Learning Representations, URL <https://openreview.net/forum?id=IPL1NIMMrw>
- [127] Wang Y, Kordi Y, Mishra S, et al (2023) Self-instruct: Aligning language models with self-generated instructions. In: The 61st Annual Meeting Of The Association For Computational Linguistics
- [128] Wei J, Wang X, Schuurmans D, et al (2022) Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems* 35:24824–24837
- [129] Weng Y, Zhu M, Xia F, et al (2023) Large language models are better reasoners with self-verification. In: Findings of the Association for Computational Linguistics: EMNLP 2023, URL <https://aclanthology.org/2023.findings-emnlp.167>
- [130] Wortsman M, Ilharco G, Gadre SY, et al (2022) Model soups: averaging weights of multiple fine-tuned models improves accuracy without increasing inference time. In: *Proceedings of the 39th International Conference on Machine Learning*, pp 23965–23998, URL <https://proceedings.mlr.press/v162/wortsman22a.html>
- [131] Wu S, Lu K, Xu B, et al (2023) Self-evolved diverse data sampling for efficient instruction tuning. arXiv preprint arXiv:231108182
- [132] Wu T, Luo L, Li YF, et al (2024) Continual learning for large language models: A survey. arXiv preprint arXiv:240201364
- [133] Xu C, Sun Q, Zheng K, et al (2024) WizardLM: Empowering large pre-trained language models to follow complex instructions. In: The Twelfth International Conference on Learning Representations, URL <https://openreview.net/forum?id=CfXh93NDgH>
- [134] Xu C, Sun Q, Zheng K, et al (2024) WizardLM: Empowering large pre-trained language models to follow complex instructions. In: The Twelfth International Conference on Learning Representations
- [135] Xu J, Zhang H, Li X, et al (2024) Sinvig: A self-evolving interactive visual agent for human-robot interaction. arXiv preprint arXiv:240211792
- [136] Xu Y, Wang S, Li P, et al (2023) Exploring large language models for communication games: An empirical study on werewolf. arXiv preprint arXiv:230904658
- [137] Yadav P, Tam D, Choshen L, et al (2023) Ties-merging: Resolving interference when merging models. In: *Advances in Neural Information Processing Systems*, pp 7093–7115, URL https://proceedings.neurips.cc/paper_files/paper/2023/file/1644c9af28ab7916874f6fd6228a9bcf-Paper-Conference.pdf
- [138] Yang K, Klein D, Celikyilmaz A, et al (2024) RLCD: Reinforcement learning from contrastive distillation for LM alignment. In: The Twelfth International Conference on Learning Representations, URL <https://openreview.net/forum?id=v3XXtxWKi6>
- [139] Yang R, Song L, Li Y, et al (2024) Gpt4tools: Teaching large language model to use tools via self-instruction. *Advances in Neural Information Processing Systems* 36
- [140] Yang Z, Li P, Liu Y (2023) Failures pave the way: Enhancing large language models through tuning-free rule accumulation. In: *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pp 1751–1777

- [141] Yang Z, Li P, Yan M, et al (2024) React meets actre: Autonomous annotations of agent trajectories for contrastive self-training. arXiv preprint arXiv:240314589
- [142] Yang Z, Liu A, Liu Z, et al (2024) Towards unified alignment between agents, humans, and environment. arXiv preprint arXiv:240207744
- [143] Yang Z, Liu Q, Pang T, et al (2024) Self-distillation bridges distribution gap in language model fine-tuning. arXiv preprint arXiv:240213669
- [144] Yao S, Zhao J, Yu D, et al (2022) React: Synergizing reasoning and acting in language models. In: The Eleventh International Conference on Learning Representations
- [145] Yao S, Yu D, Zhao J, et al (2024) Tree of thoughts: Deliberate problem solving with large language models. Advances in Neural Information Processing Systems 36
- [146] Yu L, Jiang W, Shi H, et al (2023) Metamath: Bootstrap your own mathematical questions for large language models. arXiv preprint arXiv:230912284
- [147] Yu L, Yu B, Yu H, et al (2023) Language models are super mario: Absorbing abilities from homologous models as a free lunch. arXiv preprint arXiv:231103099
- [148] Yuan W, Pang RY, Cho K, et al (2024) Self-rewarding language models. arXiv preprint arXiv:240110020
- [149] Zelikman E, Mu J, Goodman ND, et al (2022) Star: Self-taught reasoner bootstrapping reasoning with reasoning. Advances in Neural Information Processing Systems (NeurIPS)
- [150] Zelikman E, Lorch E, Mackey L, et al (2023) Self-taught optimizer (stop): Recursively self-improving code generation. arXiv preprint arXiv:231002304
- [151] Zhang C, Yang K, Hu S, et al (2024) Proagent: building proactive cooperative agents with large language models. In: Proceedings of the AAAI Conference on Artificial Intelligence, pp 17591–17599
- [152] Zhang D, Hu Z, Zhoubian S, et al (2024) Sciglm: Training scientific language models with self-reflective instruction annotation and tuning. arXiv preprint arXiv:240107950
- [153] Zhang W, Shen Y, Wu L, et al (2024) Self-contrast: Better reflection through inconsistent solving perspectives. [2401.02009](#)
- [154] Zhang W, Tang K, Wu H, et al (2024) Agent-pro: Learning to evolve via policy-level reflection and optimization. arXiv preprint arXiv:240217574
- [155] Zhang X, Peng B, Tian Y, et al (2024) Self-alignment for factuality: Mitigating hallucinations in llms via self-evaluation. arXiv preprint arXiv:240209267
- [156] Zheng H, Zhong Q, Ding L, et al (2023) Self-evolution learning for mixup: Enhance data augmentation on few-shot text classification tasks. In: Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, pp 8964–8974
- [157] Zheng L, Chiang WL, Sheng Y, et al (2024) Judging llm-as-a-judge with mt-bench and chatbot arena. Advances in Neural Information Processing Systems
- [158] Zheng T, Guo S, Qu X, et al (2024) Kun: Answer polishment for chinese self-alignment with instruction back-translation. arXiv preprint arXiv:240106477

- [159] Zhong L, Wang Z, Shang J (2024) Ldb: A large language model debugger via verifying runtime execution step-by-step. arXiv preprint arXiv:240216906
- [160] Zhong Q, Ding L, Liu J, et al (2023) Self-evolution learning for discriminative language model pretraining. In: Findings of the Association for Computational Linguistics: ACL 2023, pp 4130–4145
- [161] Zhong W, Guo L, Gao Q, et al (2024) Memorybank: Enhancing large language models with long-term memory. In: Proceedings of the AAAI Conference on Artificial Intelligence, pp 19724–19731
- [162] Zhou C, Liu P, Xu P, et al (2024) Lima: Less is more for alignment. Advances in Neural Information Processing Systems 36
- [163] Zhou X, Zhu H, Mathur L, et al (2024) SOTOPIA: Interactive evaluation for social intelligence in language agents. In: The Twelfth International Conference on Learning Representations
- [164] Zhou Z, Song J, Yao K, et al (2023) Isr-llm: Iterative self-refined large language model for long-horizon sequential task planning. arXiv preprint arXiv:230813724
- [165] Zhu Y, Qiao S, Ou Y, et al (2024) Knowagent: Knowledge-augmented planning for llm-based agents. arXiv preprint arXiv:240303101